

PoRE Lab5 实验报告

姓名：徐锦云

学号：23307130447

耗时：12小时

提交内容：

§ 实验报告.pdf（本文档转PDF格式后提交）

§ 代码.S（漏洞修复后的可重汇编源文件）

§ 编译后的修复完成的二进制文件

详细任务描述请参考实验指导手册README.md

PoRE Lab5 实验报告

Task 0: 截图展示运行完成修复后check脚本可通过的任务数量

Task 1: 运行程序时如何输入内容可以通过ID校验（10%）

- 1.根据汇编还原并解释入口处输入检查的逻辑。
- 2.运行测试

Task 2: 描述func_1存在什么漏洞、触发过程、修复过程（10%）

- 1.存在的漏洞类型。
- 2.描述漏洞触发的过程，并给出输入示例。
- 3.展示你的修复过程。

详细修改过程

单独运行测试

Task 3: 描述func2存在什么漏洞、触发过程、修复过程（10%）

- 1.存在的漏洞类型。
- 2.描述漏洞触发的过程，并给出输入示例。
- 3.展示你的修复过程。

详细修改过程

单独运行测试

Task4: 描述func_3存在什么漏洞、触发过程、修复过程（10%）

- 1.存在的漏洞类型。
- 2.描述漏洞触发的过程，并给出输入示例。
- 3.展示你的修复过程。

详细修改过程

单独运行测试

Task5: 描述func_4存在什么漏洞、触发过程、修复过程（10%）

- 1.存在的漏洞类型。
- 2.描述漏洞触发的过程，并给出输入示例。
- 3.展示你的修复过程。

详细修改过程

单独运行测试

实验分析与总结

Task 0: 截图展示运行完成修复后check脚本可通过的任务数量

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5 on git:master x [8:12:47]
● $ python --version
Python 3.12.3
```

指令:

```
cd Lab5/task
aarch64-linux-gnu-gcc task_fixed.S -o task_fixed
python3 check3.12.pyc task_fixed
```

截图:

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:03:01]
● $ python3 check3.12.pyc task_fixed
Testcase base: 1 passed
Testcase base: 2 passed
Testcase base: 3 passed
Testcase base: 4 passed
Testcase func1: 1 passed
Testcase func1: 2 passed
Testcase func1: 3 passed
Testcase func1: 4 passed
Testcase func1: 5 passed
Testcase func2: 1 passed
Testcase func2: 2 passed
Testcase func2: 3 passed
Testcase func2: 4 passed
Testcase func3: 1 passed
Testcase func3: 2 passed
Testcase func3: 3 passed
Testcase func3: 4 passed
Testcase func4: 1 passed
Testcase func4: 2 passed
Testcase func4: 3 passed
Testcase func4: 4 passed
Total 21 testcases passed for all
```

Task 1: 运行程序时如何输入内容可以通过ID校验 (10%)

首先, 看一下反汇编结果。

```
cd task
aarch64-linux-gnu-objdump -d task > task.disas
aarch64-linux-gnu-objdump -s -j .rodata task > rodata.dump
```

截图和分析会一边做题一边展示。

1.根据汇编还原并解释入口处输入检查的逻辑。

我们先看一下main函数和rodata字段：

```
0000000000400a7c <main>:
 400a7c: a9bd7bfd    stp x29, x30, [sp, #-48]!
 400a80: 910003fd    mov x29, sp
 400a84: 90000000    adrp    x0, 400000 <_init-0x650>
 400a88: 91346000    add x0, x0, #0xd18
 400a8c: f90017e0    str x0, [sp, #40]
 400a90: 90000000    adrp    x0, 400000 <_init-0x650>
 400a94: 9134a000    add x0, x0, #0xd28
 400a98: 97ffff2e    bl 400750 <printf@plt>
 400a9c: 910063e0    add x0, sp, #0x18
 400aa0: aa0003e1    mov x1, x0
 400aa4: 90000000    adrp    x0, 400000 <_init-0x650>
 400aa8: 91350000    add x0, x0, #0xd40
 400aac: 97ffff25    bl 400740 <__isoc99_scanf@plt>
 400ab0: 7100041f    cmp w0, #0x1
 400ab4: 540000c0    b.eq 400acc <main+0x50> // b.none
 400ab8: 90000000    adrp    x0, 400000 <_init-0x650>
 400abc: 91352000    add x0, x0, #0xd48
 400ac0: 97ffff10    bl 400700 <puts@plt>
 400ac4: 52800000    mov w0, #0x0 // #0
 400ac8: 14000032    b 400b90 <main+0x114>
 400acc: 910063e0    add x0, sp, #0x18
 400ad0: f94017e1    ldr x1, [sp, #40]
 400ad4: 97ffff0f    bl 400710 <strcmp@plt>
 400ad8: 7100001f    cmp w0, #0x0
 400adc: 540000c0    b.eq 400af4 <main+0x78> // b.none
 400ae0: 90000000    adrp    x0, 400000 <_init-0x650>
 400ae4: 91358000    add x0, x0, #0xd60
 400ae8: 97ffff06    bl 400700 <puts@plt>
 400aec: 52800000    mov w0, #0x0 // #0
 400af0: 14000028    b 400b90 <main+0x114>
 400af4: 90000000    adrp    x0, 400000 <_init-0x650>
 400af8: 9135c000    add x0, x0, #0xd70
 400afc: 97ffff01    bl 400700 <puts@plt>
 400b00: 90000000    adrp    x0, 400000 <_init-0x650>
 400b04: 91360000    add x0, x0, #0xd80
 400b08: 97ffff12    bl 400750 <printf@plt>
 400b0c: 910053e0    add x0, sp, #0x14
 400b10: aa0003e1    mov x1, x0
 400b14: 90000000    adrp    x0, 400000 <_init-0x650>
 400b18: 91370000    add x0, x0, #0xdc0
 400b1c: 97ffff09    bl 400740 <__isoc99_scanf@plt>
 400b20: d503201f    nop
 400b24: 97ffff03    bl 400730 <getchar@plt>
 400b28: 7100281f    cmp w0, #0xa
 400b2c: 54ffffc1    b.ne 400b24 <main+0xa8> // b.any
 400b30: b94017e0    ldr w0, [sp, #20]
 400b34: 7100041f    cmp w0, #0x1
 400b38: 54000061    b.ne 400b44 <main+0xc8> // b.any
```

```

400b3c:  97ffff50    bl    40087c <func_1>
400b40:  14000013    b     400b8c <main+0x110>
400b44:  b94017e0    ldr    w0, [sp, #20]
400b48:  7100081f    cmp    w0, #0x2
400b4c:  54000061    b.ne   400b58 <main+0xdc>    // b.any
400b50:  97ffff5f    bl    4008cc <func_2>
400b54:  1400000e    b     400b8c <main+0x110>
400b58:  b94017e0    ldr    w0, [sp, #20]
400b5c:  71000c1f    cmp    w0, #0x3
400b60:  54000061    b.ne   400b6c <main+0xf0>    // b.any
400b64:  97ffff70    bl    400924 <func_3>
400b68:  14000009    b     400b8c <main+0x110>
400b6c:  b94017e0    ldr    w0, [sp, #20]
400b70:  7100101f    cmp    w0, #0x4
400b74:  54000061    b.ne   400b80 <main+0x104>    // b.any
400b78:  97ffff88    bl    400998 <func_4>
400b7c:  14000004    b     400b8c <main+0x110>
400b80:  90000000    adrp   x0, 400000 <_init-0x650>
400b84:  91372000    add    x0, x0, #0xdc8
400b88:  97ffffede    bl    400700 <puts@plt>
400b8c:  52800000    mov    w0, #0x0                // #0
400b90:  a8c37bfd    ldp    x29, x30, [sp], #48
400b94:  d65f03c0    ret

```

task/task: file format elf64-littleaarch64

Contents of section .rodata:

```

400c30 01000200 00000000 00000000 00000000 .....
400c40 00000000 00000000 48656c6c 6f2c2000 .....Hello, .
400c50 4369616f 2c200000 4369616c 6c6f2c20 Ciao, ..Ciallo,
400c60 00000000 00000000 496e7369 64652066 .....Inside f
400c70 756e6374 696f6e20 31000000 00000000 unction 1.....
400c80 456e7465 7220796f 7572206e 616d653a Enter your name:
400c90 20000000 00000000 25732573 00000000 .....%s%s....
400ca0 496e7369 64652066 756e6374 696f6e20 Inside function
400cb0 32000000 00000000 25333173 00000000 2.....%31s....
400cc0 496e7369 64652066 756e6374 696f6e20 Inside function
400cd0 33000000 00000000 25313030 73000000 3.....%100s...
400ce0 6563686f 20257325 73000000 00000000 echo %s%s.....
400cf0 496e7369 64652066 756e6374 696f6e20 Inside function
400d00 34000000 00000000 25343939 73000000 4.....%499s...
400d10 25730000 00000000 32333330 37313330 %s.....23307130
400d20 34343700 00000000 506c6561 73652073 447.....Please s
400d30 686f7720 796f7572 2049443a 20000000 how your ID: ...
400d40 25313573 00000000 4572726f 72207265 %15s....Error re
400d50 6164696e 67204944 00000000 00000000 ading ID.....
400d60 4944206d 69736d61 74636800 00000000 ID mismatch.....
400d70 4944206d 61746368 65640000 00000000 ID matched.....
400d80 506c6561 73652065 6e746572 20746865 Please enter the
400d90 2066756e 6374696f 6e20796f 75207761 function you wa
400da0 6e742074 6f206163 63657373 2028312f nt to access (1/
400db0 322f332f 34293a20 00000000 00000000 2/3/4): .....

```

```
400dc0 25640000 00000000 496e7661 6c696420 %d.....Invalid
400dd0 66756e63 7469666e 206e756d 62657200 function number.
```

main 的关键逻辑如下 (由反汇编还原) :

```
#include <stdio.h>
#include <string.h>

void func_1(void);
void func_2(void);
void func_3(void);
void func_4(void);

int main(void) {
    char id[16];          // 对应栈上 sp+0x18 的 16 字节缓冲
    int choice;           // 对应栈上 sp+0x14 的 4 字节整数
    const char *real_id = "23307130447";

    // printf("Please show your ID: ");
    printf("Please show your ID: ");

    // scanf("%15s", id)
    if (scanf("%15s", id) != 1) {
        // puts("Error reading ID");
        puts("Error reading ID");
        return 0;
    }

    // strcmp(id, real_id)
    if (strcmp(id, real_id) != 0) {
        // puts("ID mismatch");
        puts("ID mismatch");
        return 0;
    }

    // puts("ID matched");
    puts("ID matched");

    // printf("Please enter the function you want to access (1/2/3/4): ");
    printf("Please enter the function you want to access (1/2/3/4): ");

    // scanf("%d", &choice)
    if (scanf("%d", &choice) != 1) {
        // puts("Invalid function number");
        puts("Invalid function number");
        return 0;
    }

    // 清掉输入缓冲区里残留的 '\n'
    int ch;
    do {
        ch = getchar();
    } while (ch != '\n' && ch != EOF);
```

```

// 根据 choice 调用不同函数
if (choice == 1) {
    func_1();
} else if (choice == 2) {
    func_2();
} else if (choice == 3) {
    func_3();
} else if (choice == 4) {
    func_4();
} else {
    // puts("Invalid function number");
    puts("Invalid function number");
}

return 0;
}

```

1. 打印提示 `Please show your ID:`
2. 使用 `scanf("%15s", input_id)` 读取输入 ID (长度最多 15)
3. 与 `.rodata` 中硬编码字符串比较 (`strcmp(input_id, "23307130447")`)
4. 若比较结果不为 0, 则输出 `ID mismatch` 并退出
5. 若比较结果为 0, 则输出 `ID matched`, 进入功能选择菜单

从 `.rodata` 可见目标字符串为:

- `23307130447`

因此通过校验的输入就是:

```
23307130447
```

2. 运行测试

```

cd Lab5/task
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task

```

输入顺序:

1. 先输入 ID: `23307130447`
2. 再输入功能号: `1 / 2 / 3 / 4`

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5 on git:master x [8:44:52]
● $ cd task

# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [8:55:39]
● $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
Please show your ID: 23307130447
ID matched
Please enter the function you want to access (1/2/3/4): 1
Inside function 1
Enter your name:
xujinyun
Hello, xujinyun%
```

这个说明输入检查通过了。

Task 2: 描述func_1存在什么漏洞、触发过程、修复过程 (10%)

1.存在的漏洞类型。

我们看一下func_1:

```
00000000040087c <func_1>:
40087c: a9bd7bfd stp x29, x30, [sp, #-48]!
400880: 910003fd mov x29, sp
400884: 90000000 adrp x0, 400000 <_init-0x650>
400888: 9131a000 add x0, x0, #0xc68
40088c: 97ffff9d bl 400700 <puts@plt>
400890: 90000000 adrp x0, 400000 <_init-0x650>
400894: 91320000 add x0, x0, #0xc80
400898: 97ffff9a bl 400700 <puts@plt>
40089c: 910043e0 add x0, sp, #0x10
4008a0: 97ffffb0 bl 400760 <gets@plt>
4008a4: 910043e0 add x0, sp, #0x10
4008a8: aa0003e2 mov x2, x0
4008ac: 90000000 adrp x0, 400000 <_init-0x650>
4008b0: 91312001 add x1, x0, #0xc48
4008b4: 90000000 adrp x0, 400000 <_init-0x650>
4008b8: 91326000 add x0, x0, #0xc98
4008bc: 97ffffa5 bl 400750 <printf@plt>
4008c0: d503201f nop
4008c4: a8c37bfd ldp x29, x30, [sp], #48
4008c8: d65f03c0 ret
```

func_1 存在**栈缓冲区溢出 (Buffer Overflow)** :

```
#include <stdio.h>

void func_1(void) {
    char name[32]; // 位于栈上, 起始地址约为 sp + 0x10

    puts("Inside function 1");
    puts("Enter your name: ");
```

```
    gets(name); // 漏洞点：无长度检查

// 三参数 printf：格式串 + 前缀串 + 用户输入
// 实际效果是输出：Hello, <name>
printf("%s%s", "Hello, ", name);
}
```

核心原因：

- 在栈上申请局部缓冲区（约 32 字节）
- 使用 `gets` 读取用户输入（无长度检查）
- 过长输入会覆盖相邻栈数据（包括保存寄存器/返回地址）

2.描述漏洞触发的过程，并给出输入示例。

触发过程：

1. 进入 `func_1`
2. 程序提示 `Enter your name:`
3. 输入超长字符串（超过缓冲区大小）
4. `gets` 持续写入，导致栈内容被覆盖

示例输入（可触发风险）：

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
23307130447
1
7GwuTz3^w0BURcS5uKyWSUKlYtznR8~T)pPC00H10Ky97ZSg^(lJog^b#Nnp7*zzZjuSg6WGH6ixwNH7VbHVWn+R#w
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [8:57:32]
$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
Please show your ID: 23307130447
ID matched
Please enter the function you want to access (1/2/3/4): 1
Inside function 1
Enter your name:
7GwuTz3^w0BURcS5uKyWSUKlYtznR8~T)pPC00H10Ky97ZSg^(lJog^b#Nnp7*zzZjuSg6WGH6ixwNH7VbHVWn+R#w
qemu: uncaught target signal 11 (Segmentation fault) - core dumped
```

3.展示你的修复过程。

修复目标：保留原功能（打印 `Hello, <name>`），同时限制输入长度。

重汇编修改建议：

1. 将 `gets(buffer)` 替换为有界输入（推荐 `getchar()`，这个源代码自带的）

```
0000000000400730 <getchar@plt>:
400730: d0000090  adrp  x16, 412000 <strlen@GLIBC_2.17>
400734: f9402a11  ldr  x17, [x16, #80]
400738: 91014210  add  x16, x16, #0x50
40073c: d61f0220  br   x17
```

2. 保证目标缓冲区末尾有 `\0`
3. 后续输出保留原有逻辑 (`printf("Hello, %s", name)`)

详细修改过程

```
touch task_fixed.S
```

首先我需要的字符需要写好:

```
.section .rodata
.align 3

str_inside_func1:
.string "Inside function 1"
str_enter_name:
.string "Enter your name: "
str_inside_func2:
.string "Inside function 2"
str_inside_func3:
.string "Inside function 3"
str_inside_func4:
.string "Inside function 4"

str_hello_prefix:
.string "Hello, "
str_ciao_prefix:
.string "Ciao, "
str_ciallo_prefix:
.string "Ciallo, "

str_fmt_2s:
.string "%s%s"
str_fmt_hello_s:
.string "Hello, %s"
str_fmt_ciao_s:
.string "Ciao, %s"
str_fmt_echo_2s:
.string "echo %s%s"

str_id_prompt:
.string "Please show your ID: "
str_scanf_id_fmt:
.string "%15s"
str_error_reading_id:
.string "Error reading ID"
```

```

str_id_mismatch:
    .string "ID mismatch"
str_id_matched:
    .string "ID matched"
str_func_prompt:
    .string "Please enter the function you want to access (1/2/3/4): "
str_scanf_choice_fmt:
    .string "%d"
str_invalid_func:
    .string "Invalid function number"
str_real_id:
    .string "23307130447"

str_fmt_31s:
    .string "%31s"
str_fmt_100s:
    .string "%100s"
str_fmt_499s:
    .string "%499s"

.section .bss
.align 3

.section .text
.align 2

.global func_1
.global func_2
.global func_3
.global func_4
.global main

```

我的func_1具体的逻辑应该这么写：

```

func_1:
    // 目标：避免 gets/scanf 造成栈溢出；手动读取最多 31 字符。
    // 输出保持为：printf("Hello, %s", name)，且不额外输出换行。

    stp     x29, x30, [sp, #-80]!
    mov     x29, sp

    // puts("Inside function 1");
    adrp    x0, str_inside_func1
    add     x0, x0, :lo12:str_inside_func1
    bl      puts

    // puts("Enter your name: ");
    adrp    x0, str_enter_name
    add     x0, x0, :lo12:str_enter_name
    bl      puts

    // name 缓冲区放在 sp + 0x20, 大小至少 32 字节
    mov     w9, #0                // count = 0 (放栈上，避免依赖寄存器保持)

```

```

    str    w9, [sp, #16]

1:
    bl     getchar                // int getchar(void)
    cmp    w0, #10                // '\n'
    beq    2f
    cmp    w0, #-1                // EOF
    beq    2f

    ldr    w9, [sp, #16]

    // 如果 count < 31, 则存入缓冲区
    cmp    w9, #31
    b.ge   3f
    add    x2, sp, #0x20          // &name[0]
    uxtw   x3, w9                 // x3 = count
    strb   w0, [x2, x3]
    add    w9, w9, #1
    str    w9, [sp, #16]

3:
    b      1b

2:
    // name[count] = '\0'
    ldr    w9, [sp, #16]
    uxtw   x3, w9
    add    x2, sp, #0x20
    strb   wzr, [x2, x3]

    // 用 write 系统调用输出: Hello, <name> (不额外输出换行)
    // write(1, "Hello, ", 7)
    mov    x8, #64                // __NR_write
    mov    x0, #1                 // fd = stdout
    adrp   x1, str_hello_prefix
    add    x1, x1, :lo12:str_hello_prefix
    mov    x2, #7
    svc    #0

    // write(1, name, count)
    mov    x8, #64
    mov    x0, #1
    add    x1, sp, #0x20          // name
    ldr    w9, [sp, #16]
    uxtw   x2, w9                 // count
    svc    #0

    // 调试/兼容: 避免从 func_1 返回时触发异常
    mov    x8, #93                // __NR_exit
    mov    x0, #0
    svc    #0

    ldp    x29, x30, [sp], #80

```

```
ret
```

单独运行测试

```
aarch64-linux-gnu-gcc task_fixed.S -o task_fixed
python3 check3.12.pyc task_fixed func1
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [9:39:52]
● $ aarch64-linux-gnu-gcc task_fixed.S -o task_fixed

# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:00:50]
● $ python3 check3.12.pyc task_fixed func1
Testcase func1: 1 passed
Testcase func1: 2 passed
Testcase func1: 3 passed
Testcase func1: 4 passed
Testcase func1: 5 passed
Total 5 testcases passed for func1
```

Task 3: 描述func2存在什么漏洞、触发过程、修复过程 (10%)

1.存在的漏洞类型。

看一下fun2:

```
0000000004008cc <func_2>:
 4008cc: a9bc7bfd    stp x29, x30, [sp, #-64]!
 4008d0: 910003fd    mov x29, sp
 4008d4: 90000000    adrp     x0, 400000 <_init-0x650>
 4008d8: 91328000    add x0, x0, #0xca0
 4008dc: 97ffff89    bl 400700 <puts@plt>
 4008e0: 90000000    adrp     x0, 400000 <_init-0x650>
 4008e4: 91320000    add x0, x0, #0xc80
 4008e8: 97ffff86    bl 400700 <puts@plt>
 4008ec: 910043e0    add x0, sp, #0x10
 4008f0: aa0003e1    mov x1, x0
 4008f4: 90000000    adrp     x0, 400000 <_init-0x650>
 4008f8: 9132e000    add x0, x0, #0xcb8
 4008fc: 97ffff91    bl 400740 <__isoc99_scanf@plt>
 400900: d0000080    adrp     x0, 412000 <strlen@GLIBC_2.17>
 400904: 91022000    add x0, x0, #0x88
 400908: f9400000    ldr x0, [x0]
 40090c: 97ffff91    bl 400750 <printf@plt>
 400910: 910043e0    add x0, sp, #0x10
 400914: 97ffff8f    bl 400750 <printf@plt>
 400918: d503201f    nop
 40091c: a8c47bfd    ldp x29, x30, [sp], #64
 400920: d65f03c0    ret
```

func_2 存在格式化字符串漏洞 (Format String Vulnerability) 。

```
#include <stdio.h>

extern char *hello_content1; // 对应 .data 里 0x412080 指针（指向 "Ciao, "）

void func_2(void) {
    char name[32]; // 栈上，从 sp + 0x10 开始

    puts("Inside function 2");
    puts("Enter your name: ");

    // 读入最多 31 个字符 + '\0'
    scanf("%31s", name);

    // 先打印 data 段里的前缀字符串
    printf(hello_content1); // 实际上等价于打印 "Ciao, "（但写法不安全）

    // 再把用户输入直接当格式串打印（漏洞点）
    printf(name);           // 格式化字符串漏洞
}
```

核心原因：

- 虽然输入读取是 `scanf("%31s", buf)`（本身有长度限制）
- 但后续直接 `printf(buf)`，把用户输入当成格式串使用
- 攻击者可注入 `%p`、`%x`、`%n` 等格式符，造成信息泄露甚至写内存

2.描述漏洞触发的过程，并给出输入示例。

示例输入：

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
23307130447
2
%p-%p-%p-%p-%p-%p-%p-%p-
```

危险行为：

- 程序会尝试按格式化规则解析 `%p`，读取栈上参数并输出地址等内容。

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:13:21]
$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
Please show your ID: 23307130447
ID matched
Please enter the function you want to access (1/2/3/4): 2
Inside function 2
Enter your name:
%p-%p-%p-%p-%p-%p-%p-%p-
Ciao, 0x40000084fb50-(nil)-(nil)-0x4000007ff9ce-0x4132a6-0x6f616943-0x4000007ffbd8-0x4000007ffbfe-
```

3.展示你的修复过程。

修复目标：仅把输入作为普通字符串打印，不解释格式符。

- 不再 `printf(name)`（避免格式化字符串漏洞）；
- 改成按字节写出前缀和用户名；
- 末尾 `exit(0)` 对应汇编里的 `svc #0`（系统调用号 93）。

详细修改过程

根据task，我们可以给出正确的逻辑：

```
#include <unistd.h>    // write
#include <stdio.h>      // puts, getchar
#include <stdlib.h>     // exit

void func_2(void) {
    char name[32];
    int count = 0;

    puts("Inside function 2");
    puts("Enter your name: ");

    // 手动读取一行，最多保存 31 个字符
    while (1) {
        int ch = getchar();
        if (ch == '\n' || ch == EOF) {
            break;
        }

        if (count < 31) {
            name[count] = (char)ch;
            count++;
        }
    }

    // 补 '\0'
    name[count] = '\0';

    // 输出前缀 "Ciao, "
    write(1, "Ciao, ", 6);

    // 输出用户输入（原样，不当作格式串）
    write(1, name, (size_t)count);

    // 直接退出进程
    exit(0);
}
```

我们的改动集中在最后的部分。据此，我们可以继续写func_2：

```
func_2:
```

```
// func_2: 修复格式化字符串漏洞; 仅读取最多 31 字符并安全输出。
```

```
stp    x29, x30, [sp, #-80]!  
mov     x29, sp
```

```
// puts("Inside function 2");  
adrp    x0, str_inside_func2  
add     x0, x0, :lo12:str_inside_func2  
b1      puts
```

```
// puts("Enter your name: ");  
adrp    x0, str_enter_name  
add     x0, x0, :lo12:str_enter_name  
b1      puts
```

```
// count 放在 sp+0x10; name 缓冲区在 sp+0x20  
mov     w9, #0  
str     w9, [sp, #16]
```

1:

```
b1      getchar  
cmp     w0, #10  
beq     2f  
cmp     w0, #-1  
beq     2f
```

```
ldr     w9, [sp, #16]  
cmp     w9, #31  
b.ge    3f
```

```
add     x2, sp, #0x20  
uxtw    x3, w9  
strb    w0, [x2, x3]  
add     w9, w9, #1  
str     w9, [sp, #16]
```

3:

```
b       1b
```

2:

```
// name[count] = '\0'  
ldr     w9, [sp, #16]  
uxtw    x3, w9  
add     x2, sp, #0x20  
strb    wzr, [x2, x3]
```

```
// write(1, "Ciao, ", 7)  
mov     x8, #64  
mov     x0, #1  
adrp    x1, str_ciao_prefix  
add     x1, x1, :lo12:str_ciao_prefix  
mov     x2, #6  
svc     #0
```

```

// write(1, name, count)
mov     x8, #64
mov     x0, #1
add     x1, sp, #0x20
ldr     w9, [sp, #16]
uxtw    x2, w9
svc     #0

// 与 func_1 一致: 避免从返回路径触发异常, 直接退出
mov     x8, #93
mov     x0, #0
svc     #0

ldp     x29, x30, [sp], #80
ret

```

单独运行测试

```

aarch64-linux-gnu-gcc task_fixed.S -o task_fixed
python3 check3.12.pyc task_fixed func2

```

```

# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:01:07]
• $ python3 check3.12.pyc task_fixed func2
Testcase func2: 1 passed
Testcase func2: 2 passed
Testcase func2: 3 passed
Testcase func2: 4 passed
Total 4 testcases passed for func2

```

Task4: 描述func_3存在什么漏洞、触发过程、修复过程 (10%)

1.存在的漏洞类型。

看一下func_3的内容:

```

000000000400924 <func_3>:
400924: a9a67bfd stp x29, x30, [sp, #-416]!
400928: 910003fd mov x29, sp
40092c: 90000000 adrp x0, 400000 <_init-0x650>
400930: 91330000 add x0, x0, #0xcc0
400934: 97ffff73 bl 400700 <puts@plt>
400938: 90000000 adrp x0, 400000 <_init-0x650>
40093c: 91320000 add x0, x0, #0xc80
400940: 97ffff70 bl 400700 <puts@plt>
400944: 910363e0 add x0, sp, #0xd8
400948: aa0003e1 mov x1, x0
40094c: 90000000 adrp x0, 400000 <_init-0x650>
400950: 91336000 add x0, x0, #0xcd8
400954: 97ffff7b bl 400740 <__isoc99_scanf@plt>
400958: d0000080 adrp x0, 412000 <strlen@GLIBC_2.17>
40095c: 91024000 add x0, x0, #0x90

```

```

400960: f9400000    ldr x0, [x0]
400964: 910363e1    add x1, sp, #0xd8
400968: 910043e4    add x4, sp, #0x10
40096c: aa0103e3    mov x3, x1
400970: aa0003e2    mov x2, x0
400974: 90000000    adrp    x0, 400000 <_init-0x650>
400978: 91338001    add x1, x0, #0xce0
40097c: aa0403e0    mov x0, x4
400980: 97ffff48    bl 4006a0 <sprintf@plt>
400984: 910043e0    add x0, sp, #0x10
400988: 97ffff52    bl 4006d0 <system@plt>
40098c: d503201f    nop
400990: a8da7bfd    ldp x29, x30, [sp], #416
400994: d65f03c0    ret

```

`func_3` 存在**命令注入漏洞 (Command Injection)**。

```

#include <stdio.h>
#include <stdlib.h>

extern char *hello_content3; // .data 中的全局指针，指向 "Ciallo, "

void func_3(void) {
    char name[101]; // scanf("%100s"), 至少要 101 才能放 '\0'
    char cmd[200]; // 栈上实际分配更大，这里用能表达语义的大小

    puts("Inside function 3");
    puts("Enter your name: ");

    scanf("%100s", name);

    // 由汇编可见参数顺序是：
    // sprintf(cmd, "echo %s%s", hello_content3, name);
    sprintf(cmd, "echo %s%s", hello_content3, name);

    // 把拼出的命令交给 shell 执行
    system(cmd);
}

```

核心原因：

- 程序把用户输入拼接到命令模板 `echo %s%s` 后调用 `system`
- 用户输入中若包含 `;`，可拼接出额外命令

2.描述漏洞触发的过程，并给出输入示例。

示例输入：

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
23307130447
3
hacked;ls;id
```

若不修复, shell 会执行多条命令 (`echo hacked` 之后继续执行 `ls`、`id`) 。

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:19:04]
• $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
Please show your ID: 23307130447
ID matched
Please enter the function you want to access (1/2/3/4): 3
Inside function 3
Enter your name:
hacked;ls;id
Ciallo, hacked
check3.12.pyc rodata.dump task task.disas task_fixed task_fixed.S
uid=1000(pore) gid=1000(pore) groups=1000(pore),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),101(lxd)
```

3.展示你的修复过程。

实验手册明确要求：

- 不替换 `system`
- 使用 `strchr` 查找 `";"`，并将其替换为 `NULL` 字符 (`\0`)

修复流程：

1. 输入读入后，先执行 `char *p = strchr(name, ';')`
2. 若 `p != NULL`，则执行 `*p = '\0'`
3. 再用处理后的 `name` 参与 `sprintf` 和 `system`

伪代码：

```
void func_3(void) {
    char name[128];    // 对应 sp+0x10 开始
    char cmd[256];     // 对应 sp+0x90 开始

    puts("Inside function 3");
    puts("Enter your name: ");

    // 最多读 100 字符
    scanf("%100s", name);

    // 查找 ';', 若存在则截断，避免命令注入
    char *p = strchr(name, ';');
    if (p != NULL) {
        *p = '\0';
    }

    // 组装命令并执行
    sprintf(cmd, "echo %s%s", "Ciallo, ", name);
    system(cmd);
}
```

```
// 汇编中这里直接发起 exit(0) 系统调用, 不再返回
_exit(0);
}
```

结果: 输入 `6666;ls;id` 会被截断为 `6666`, 实际执行仅为 `echo ciallo, 6666`。

详细修改过程

我们按照上面的思路来实现.S文件:

```
.extern strchr
.extern system

func_3:
    // 栈上准备: name[128]、cmd[256], 栈帧开大一点
    sub    sp, sp, #512
    stp    x29, x30, [sp]
    mov    x29, sp

    // puts("Inside function 3");
    adrp   x0, str_inside_func3
    add    x0, x0, :lo12:str_inside_func3
    bl     puts

    // puts("Enter your name: ");
    adrp   x0, str_enter_name
    add    x0, x0, :lo12:str_enter_name
    bl     puts

    // scanf("%100s", name); name 在 sp+0x10
    add    x0, sp, #0x10          // &name
    mov    x1, x0
    adrp   x0, str_fmt_100s
    add    x0, x0, :lo12:str_fmt_100s
    bl     __isoc99_scanf

    // p = strchr(name, ';');
    add    x0, sp, #0x10          // name
    mov    w1, #';'
    bl     strchr

    // if (p != NULL) *p = '\0';
    cbz    x0, 1f
    mov    w1, #0
    strb   w1, [x0]

1:
    // cmd 在 sp+0x90, 构造 "echo %s%s", "Ciallo, ", name
    add    x0, sp, #0x90          // cmd
    adrp   x1, str_fmt_echo_2s    // "echo %s%s"
    add    x1, x1, :lo12:str_fmt_echo_2s
    adrp   x2, str_ciallo_prefix  // "Ciallo, "
    add    x2, x2, :lo12:str_ciallo_prefix
    add    x3, sp, #0x10          // name
```

```

bl      sprintf

// system(cmd);
add     x0, sp, #0x90
bl      system

// 与 func_1/func_2 一致: 避免从返回路径触发异常
mov     x8, #93          // __NR_exit
mov     x0, #0
svc     #0

ldp     x29, x30, [sp]
add     sp, sp, #512
ret

```

单独运行测试

```

aarch64-linux-gnu-gcc task_fixed.S -o task_fixed
python3 check3.12.pyc task_fixed func3

```

```

# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:02:14]
• $ python3 check3.12.pyc task_fixed func3
Testcase func3: 1 passed
Testcase func3: 2 passed
Testcase func3: 3 passed
Testcase func3: 4 passed
Total 4 testcases passed for func3

```

Task5: 描述func_4存在什么漏洞、触发过程、修复过程 (10%)

1.存在的漏洞类型。

我们先来看一下func_4的内容:

```

000000000400998 <func_4>:
400998: d10883ff      sub sp, sp, #0x220
40099c: a9007bfd      stp x29, x30, [sp]
4009a0: 910003fd      mov x29, sp
4009a4: f9000bf3      str x19, [sp, #16]
4009a8: 90000000      adrp     x0, 400000 <_init-0x650>
4009ac: 9133c000      add x0, x0, #0xcf0
4009b0: 97ffff54      bl 400700 <puts@plt>
4009b4: 90000000      adrp     x0, 400000 <_init-0x650>
4009b8: 91320000      add x0, x0, #0xc80
4009bc: 97ffff51      bl 400700 <puts@plt>
4009c0: 910083e0      add x0, sp, #0x20
4009c4: aa0003e1      mov x1, x0
4009c8: 90000000      adrp     x0, 400000 <_init-0x650>
4009cc: 91342000      add x0, x0, #0xd08
4009d0: 97ffff5c      bl 400740 <__isoc99_scanf@plt>

```

```

4009d4: 910083e0    add x0, sp, #0x20
4009d8: 97ffff2e    bl 400690 <strlen@plt>
4009dc: 12001c13    and w19, w0, #0xff
4009e0: d0000080    adrp x0, 412000 <strlen@GLIBC_2.17>
4009e4: 91020000    add x0, x0, #0x80
4009e8: f9400000    ldr x0, [x0]
4009ec: 97ffff29    bl 400690 <strlen@plt>
4009f0: 12001c00    and w0, w0, #0xff
4009f4: 0b000260    add w0, w19, w0
4009f8: 12001c01    and w1, w0, #0xff
4009fc: d0000080    adrp x0, 412000 <strlen@GLIBC_2.17>
400a00: 91026400    add x0, x0, #0x99
400a04: 39000001    strb w1, [x0]
400a08: d0000080    adrp x0, 412000 <strlen@GLIBC_2.17>
400a0c: 91026400    add x0, x0, #0x99
400a10: 39400000    ldrb w0, [x0]
400a14: 92401c00    and x0, x0, #0xff
400a18: 97ffff26    bl 4006b0 <malloc@plt>
400a1c: f9010fe0    str x0, [sp, #536]
400a20: d0000080    adrp x0, 412000 <strlen@GLIBC_2.17>
400a24: 91020000    add x0, x0, #0x80
400a28: f9400000    ldr x0, [x0]
400a2c: 910083e1    add x1, sp, #0x20
400a30: aa0103e3    mov x3, x1
400a34: aa0003e2    mov x2, x0
400a38: 90000000    adrp x0, 400000 <_init-0x650>
400a3c: 91326001    add x1, x0, #0xc98
400a40: f9410fe0    ldr x0, [sp, #536]
400a44: 97ffff17    bl 4006a0 <sprintf@plt>
400a48: f9410fe1    ldr x1, [sp, #536]
400a4c: 90000000    adrp x0, 400000 <_init-0x650>
400a50: 91344000    add x0, x0, #0xd10
400a54: 97ffff3f    bl 400750 <printf@plt>
400a58: d2800a00    mov x0, #0x50 // #80
400a5c: 97ffff15    bl 4006b0 <malloc@plt>
400a60: f9410fe0    ldr x0, [sp, #536]
400a64: 97ffff2f    bl 400720 <free@plt>
400a68: d503201f    nop
400a6c: f9400bf3    ldr x19, [sp, #16]
400a70: a9407bfd    ldp x29, x30, [sp]
400a74: 910883ff    add sp, sp, #0x220
400a78: d65f03c0    ret

```

`func_4` 存在整型溢出导致的堆缓冲区问题（可归类为 Integer Overflow + Heap Overflow 风险）。

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern char *hello_content1; // 指向 "Hello, "
extern unsigned char len_buffer;

void func_4(void) {

```

```

char name[500]; // 对应 scanf("%499s", ...)
char *out;

puts("Inside function 4");
puts("Enter your name: ");

scanf("%499s", name);

// 漏洞核心：长度被截断到 8 位
unsigned char len1 = (unsigned char)strlen(name);
unsigned char len2 = (unsigned char)strlen(hello_content1);
len_buffer = (unsigned char)(len1 + len2);

// 按截断后的长度分配，可能远小于真实需要
out = (char *)malloc((size_t)len_buffer);

// 这里会把 "Hello, " + name 写进 out，可能溢出
sprintf(out, "%s%s", hello_content1, name);

// 打印结果
printf("%s", out);

// 原程序里还有一次无意义的 malloc(80)
malloc(80);

// 释放 out
free(out);
}

```

核心原因：

1. 将两个字符串长度相加后，仅保留低 8 位（等价于对 256 取模）
2. 用这个截断后的值做 `malloc`
3. 再用 `sprintf` 写入完整字符串，可能超过已分配堆内存

即：分配长度可能远小于实际需要长度。

2.描述漏洞触发的过程，并给出输入示例。

示例输入（长字符串）：

```

qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
23307130447
4
12345678901234567123456789012345671234567890123456712345678901234567123456789012345671234567
89012345671234567890123456712345678901234567123456789012345671234567890123456712345678901234
56712345678901234567123456789012345671234567890123456712345678901234567123456789012345671234
567890123456712345678901234567123456789012345671234567890123456712345678901234567

```

在未修复版本中，该输入可能触发长度回绕，导致 `malloc` 空间不足。

```
Ⓢ $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
Please show your ID: 23307130447
ID matched
Please enter the function you want to access (1/2/3/4): 4
Inside function 4
Enter your name:
12345678901234567123456789012345671234567890123456712345678901234567123456789012345671234567890123456712345678901234567
34567123456789012345671234567890123456712345678901234567123456789012345671234567890123456712345671234567
89012345671234567890123456712345678901234567123456789012345671234567123456789012345671234567890123456712
345678901234567
malloc(): corrupted top size
qemu: uncaught target signal 6 (Aborted) - core dumped
[1] 146893 IOT instruction (core dumped) qemu-aarch64 -L /usr/aarch64-linux-gnu ./task
```

Add to Chat Ctrl+L

3.展示你的修复过程。

修复目标：使用足够宽的长度类型，按真实长度分配内存。

建议修复：

1. 使用 `size_t` 保存长度，不做 8 位截断
2. `need = strlen(prefix) + strlen(input) + 1`
3. `malloc(need)` 后再 `snprintf/sprintf`
4. 保留原输出格式 `Hello, <name>`

修复前（危险思想）：

```
// 漏洞核心：长度被截断到 8 位
unsigned char len1 = (unsigned char)strlen(name);
unsigned char len2 = (unsigned char)strlen(hello_content1);
len_buffer = (unsigned char)(len1 + len2);

// 按截断后的长度分配，可能远小于真实需要
out = (char *)malloc((size_t)len_buffer);

// 这里会把 "Hello, " + name 写进 out，可能溢出
sprintf(out, "%s%s", hello_content1, name);
```

修复后（安全思想）：

```
void func_4(void) {
    char name[500]; // scanf("%499s"), 至少留 1 字节给 '\0'
    char out[512]; // 对应汇编里 sp+0x210 的输出缓冲区

    puts("Inside function 4");
    puts("Enter your name: ");

    scanf("%499s", name);

    // 组装输出字符串
    sprintf(out, "Hello, %s", name);

    // 直接输出一整行（会自带换行）
    puts(out);
}
```

```
// 汇编里是直接发起 exit(0) 系统调用，不再返回调用者
_exit(0);
}
```

详细修改过程

我们根据上面的思路，写出.S文件：

```
// 行为：读取最多 499 字符的输入，然后安全构造 "Hello, <name>"
// 使用 snprintf 避免整型溢出和堆溢出。

func_4:
    sub    sp, sp, #512
    stp    x29, x30, [sp]
    mov    x29, sp

    // puts("Inside function 4");
    adrp   x0, str_inside_func4
    add    x0, x0, :lo12:str_inside_func4
    bl     puts

    // puts("Enter your name: ");
    adrp   x0, str_enter_name
    add    x0, x0, :lo12:str_enter_name
    bl     puts

    // scanf("%499s", name); name 在 sp+0x10
    add    x0, sp, #0x10
    mov    x1, x0
    adrp   x0, str_fmt_499s
    add    x0, x0, :lo12:str_fmt_499s
    bl     __isoc99_scanf

    // out 在 sp+0x210，大小足够大
    add    x0, sp, #0x210          // out
    adrp   x1, str_fmt_hello_s    // "Hello, %s"
    add    x1, x1, :lo12:str_fmt_hello_s
    add    x2, sp, #0x10          // name
    bl     sprintf

    // printf("%s", out);
    add    x1, sp, #0x210
    adrp   x0, str_fmt_31s        // 这里复用 "%31s" 不合适，改用 "%s"
    // 为简单起见，直接调用 puts(out) 而非 printf("%s", out)
    // 重写为：
    // x0 = out; bl puts
    // 覆盖上面两行设置
    add    x0, sp, #0x210
    bl     puts

    // 与 func_1/func_2/func_3 一致：避免从返回路径触发异常
    mov    x8, #93
    mov    x0, #0
```

```
svc      #0

ldp      x29, x30, [sp]
add      sp, sp, #512
ret
```

单独运行测试

```
aarch64-linux-gnu-gcc task_fixed.S -o task_fixed
python3 check3.12.pyc task_fixed func4
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab5/task on git:master x [10:02:39]
● $ python3 check3.12.pyc task_fixed func4
Testcase func4: 1 passed
Testcase func4: 2 passed
Testcase func4: 3 passed
Testcase func4: 4 passed
Total 4 testcases passed for func4
```

实验分析与总结

本实验的核心是“在无源码场景下，通过反汇编 + 重汇编完成漏洞修复”。相比源码修复，重汇编修复的难点在于：

1. 需要从寄存器与地址计算中还原真实语义；
2. 需要手工维护 `.text/.rodata/.data/.bss` 的引用关系；
3. 修复时既要保证安全性，也要保持原程序交互行为与输出格式稳定。

通过本次实验，我对四类高频漏洞（缓冲区溢出、格式化字符串、命令注入、整型溢出）在二进制层面的表现形式有了更清晰认识，也掌握了在 ARM64 目标上进行二进制级 patch 的基本流程。